Software Architecture and Design Specification

Event Management and Registration System (EMRS)

Project: Event Management and Registration System (EMRS)

Version: 1.1

Authors: Aariz, Dhanraj Rathod, Vaishali Krishnan, Avinash Julakatti

Date: 02-10-2026

Status: Submission Version

Revision History

Version	Date	Author	Change Summary
1.1	02-10-2026	EMRS Project Team	Updated component architecture, component descriptions, interfaces, security mapping, and traceability to align with the submitted UML component diagram.

Approvals

Role	Name	Signature / Date
Project Manager	Aariz	
Development Lead	Dhanraj Rathod	
Product Owner	Vaishali Krishnan	
QA / Tester	Avinash Julakatti	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Event Management and Registration System (EMRS) v1.0. It translates the requirements of the Software Requirements Specification (SRS v1.0, 03-09-2026) into an architecture consistent with the EMRS UML component diagram: UI/Presentation Layer, Backend REST API Gateway, Business Logic Layer, Data Access Layer, external service interfaces, and the relational database. It also provides detailed design (sequence diagrams, API definitions, error handling, UX) that developers can implement and QA engineers can verify against the Software Test Plan (STP v1.0).

1.2 Scope

Covers the architecture and design of every capability in the SRS scope: user accounts (registration, login, password reset, profile); organizer event creation, editing and cancellation; attendee discovery, search and registration including waitlisting; simulated payment for paid events; e-mail notifications; QR e-tickets and check-in; organizer dashboard and CSV reporting; platform administration and moderation; post-event feedback; and the non-functional and security requirements (EMRS-NF-001 to 005, EMRS-SEC-001 to 005).

Excludes venue physical infrastructure and the internal implementation of third-party payment, e-mail and maps providers beyond their published APIs (SRS §1.2).

1.3 Audience

Developers, QA engineers, security auditors, event organizers and system administrators (as reviewers of intended behaviour), instructors/evaluators, and maintenance teams.

1.4 Definitions

Term	Meaning
EMRS	Event Management and Registration System
SRS / SAD / STP	Software Requirements Specification / Software Architecture and Design Specification (this document) / Software Test Plan
RTM	Requirements Traceability Matrix (SRS §8, STP §13, and §3.8 of this document)
UML	Unified Modeling Language
ADR	Architecture Decision Record - a short record of one significant design decision, its context and consequences
API / REST / JSON	Application Programming Interface / Representational State Transfer / JavaScript Object Notation
SPA	Single-Page Application (browser front-end)
JWT	JSON Web Token - signed token used to authenticate requests after login
RBAC	Role-Based Access Control (Attendee, Organizer, Admin)

Term	Meaning
TLS / HTTPS	Transport Layer Security / HTTP over TLS
QR	Quick Response code, used on e-tickets for check-in
CAPTCHA	Challenge-response test used to block automated (bot) abuse
STRIDE	Threat classification: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
PCI-DSS	Payment Card Industry Data Security Standard
WCAG 2.1 AA	Web Content Accessibility Guidelines, version 2.1, conformance level AA
CSV	Comma-Separated Values (export format for organizer reports)

2. Document Overview

2.1 How to use this document

This document provides the architectural deliverables required for EMRS. Use it as follows:

- **UML diagrams** - component diagram and domain class diagram (§3.3), deployment view (§3.6), four sequence diagrams (§4.2).
- **Architecture decision records (ADRs)** - seven recorded decisions with context and consequences (§3.5).
- **Threat model** - STRIDE analysis and security controls mapped to EMRS-SEC-001 to 005 (§3.9).
- **API design** - conventions, endpoint catalog, detailed interface definitions and internal interfaces (§4.3).
- **Traceability** - every SRS requirement mapped to components, design sections and STP test cases (§3.8).

Suggested reading: developers - §3.3 to 3.6 then §4; QA engineers - §3.8, §4.3, §4.4 together with the STP; security auditors - §3.9 and §4.4; evaluators - §3.8 first, then the sections it references.

2.2 Related Documents

Document	Version / Date	Relationship to this SAD
Software Requirements Specification (SRS)	v1.0 / 03-09-2026	Source of all requirement IDs (EMRS-F-001 to 019, EMRS-NF-001 to 005, EMRS-SEC-001 to 005).
Software Test Plan (STP)	v1.0 / 02-10-2026	Defines the test cases (TC-*) that verify this design; referenced in §3.8.
Requirements Traceability Matrix (RTM)	SRS §8 / STP §13	Requirement-to-test mapping. §3.8 extends it with design components and design references.

3. Architecture

3.1 Goals & Constraints

Architectural goals - derived from the SRS quality requirements:

ID	Goal	Architectural response	SRS source
G1	Security	Defence in depth: TLS, filter chain (rate limit, CAPTCHA, JWT, RBAC), bcrypt hashing, tokenised payments, audit logging.	SEC-001 to 005
G2	Performance	95% of page requests in under 2 s: indexed queries, pagination, caching of read-mostly data, connection pooling, asynchronous e-mail.	NF-001
G3	Availability	Available throughout scheduled demonstration and testing periods: health probes, automatic restart, database backups, graceful degradation when third parties fail.	NF-002
G4	Scalability	Stateless back-end (JWT) that can be replicated behind a load balancer; short database transactions; asynchronous work off the request path.	NF-003
G5	Usability & accessibility	Responsive SPA for mobile, tablet and desktop; WCAG 2.1 AA; simple multi-step checkout.	NF-004, §3.1
G6	Maintainability	Layered, package-by-feature modules with interfaces for every external dependency; automated unit and integration tests.	NF-005
G7	Correctness of registration	Capacity is never oversold and duplicate registrations are impossible, even under concurrent requests.	F-010, F-012

Constraints:

ID	Constraint	Impact on the design
C1	Card data must never be stored; payment handling is delegated to a PCI-DSS compliant gateway (SEC-005, SRS §2.5).	Only gateway tokens and transaction references are persisted; Payment module talks to a gateway adapter.
C2	Payment processing is simulated for this release (F-013).	Simulated gateway implementation behind the same interface a real gateway would use.
C3	Depends on internet connectivity and third-party e-mail services (SRS §2.5).	Notifications are asynchronous with retry; failures never block registration.
C4	All client-server traffic over TLS 1.2 or higher (SEC-002, SRS §3.4).	TLS terminated at the reverse proxy; HTTP redirected to HTTPS; HSTS enabled.
C5	Relational database (SRS §2.1).	MySQL with transactions and constraints enforcing capacity and uniqueness.
C6	Modern desktop and mobile browsers (SRS §2.4); optional camera device for QR scanning (SRS §3.2).	Browser-based scanner using the device camera; manual ticket-code entry as fallback.
C7	Small student team and fixed schedule.	A single deployable application is preferred over distributed services (ADR-001).

3.2 Stakeholders & Concerns

Stakeholder	Concerns	Addressed in
Attendees	Fast, mobile-friendly registration; safety of personal and payment data; dependable e-ticket and e-mails.	§4.5, §3.9, ADR-005, ADR-006
Organizers	Clear dashboard; reliable notifications; correct capacity; quick check-in; only they can change their events.	Event, Reporting and Check-in modules; RBAC (§3.9)
Admins	Platform-wide moderation; policy enforcement; accountability of actions.	Admin module; audit log (§4.4)
Developers	Modularity, clear interfaces, testability, low coupling to third parties.	§3.5, §3.4 (adapters), §4.3.4
QA engineers	Testable requirements, traceability, sandbox/simulated external services.	§3.8, ADR-005, §4.4
Security auditors / regulators	Password storage, TLS, RBAC, no card data held, PCI-DSS responsibility boundary.	§3.9
Product Owner / Project Manager	Scope coverage versus SRS, risk, delivery on time.	§3.7, §3.8
Instructors / evaluators	Completeness against the SRS and template; traceability.	§2, §3.8
Third-party providers (e-mail, payment, maps, CAPTCHA)	Stable and minimal integration boundary using published APIs only.	Integration adapters (§3.4)

3.3 Component (UML) Diagram

Figure 3.1 shows the logical components and interfaces of EMRS as represented in the submitted UML component diagram. The architecture is organised into four visible areas: (1) the UI / Presentation Layer containing the Web Client and QR Check-in Scanner; (2) the Backend REST API Gateway, which is the common REST entry point and enforces HTTPS/TLS, JWT validation, RBAC, rate limiting and CAPTCHA; (3) the Business Logic Layer containing the feature modules; and (4) the Data Access Layer and external service interfaces. The feature modules are EventModule, AuthModule, RegistrationModule, CheckInModule, ReportingModule, AdminModule, FeedbackModule, NotificationModule, PaymentModule and TicketModule. Profile operations are part of AuthModule in this diagram. External integrations are represented by the Maps / Geocoding API, Email/SMS Provider and Payment Gateway API. The Data Access Layer connects to the Relational Database through SQL/JPA. The interface labels in Figure 3.1 (for example RESTAPI, IAuth, IRegistration, ICheckIn, IReporting, IModeration, IFeedback,

INotification, IPayment, ITicket, IRepository and ISQL) are the architectural contracts between these components.

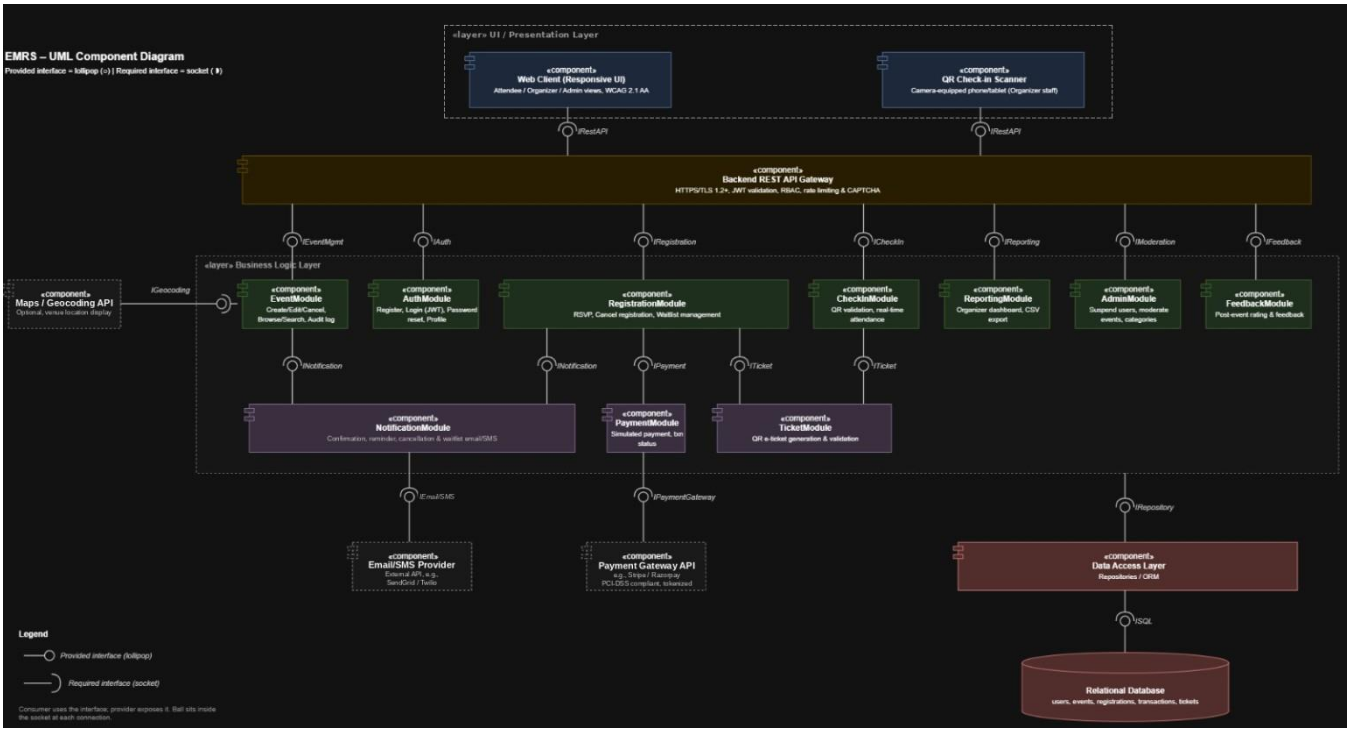


Figure 3.1 - EMRS UML component diagram (submitted architecture view)

Figure 3.2 shows the domain model (UML class diagram) that the modules operate on. It also defines the structure of the MySQL schema (§3.6). Notable design points: a Registration owns at most one

Payment and one Ticket; a Ticket can be checked in at most once; every change to an Event is recorded in an AuditLog; waitlisting is modelled explicitly with WaitlistEntry.

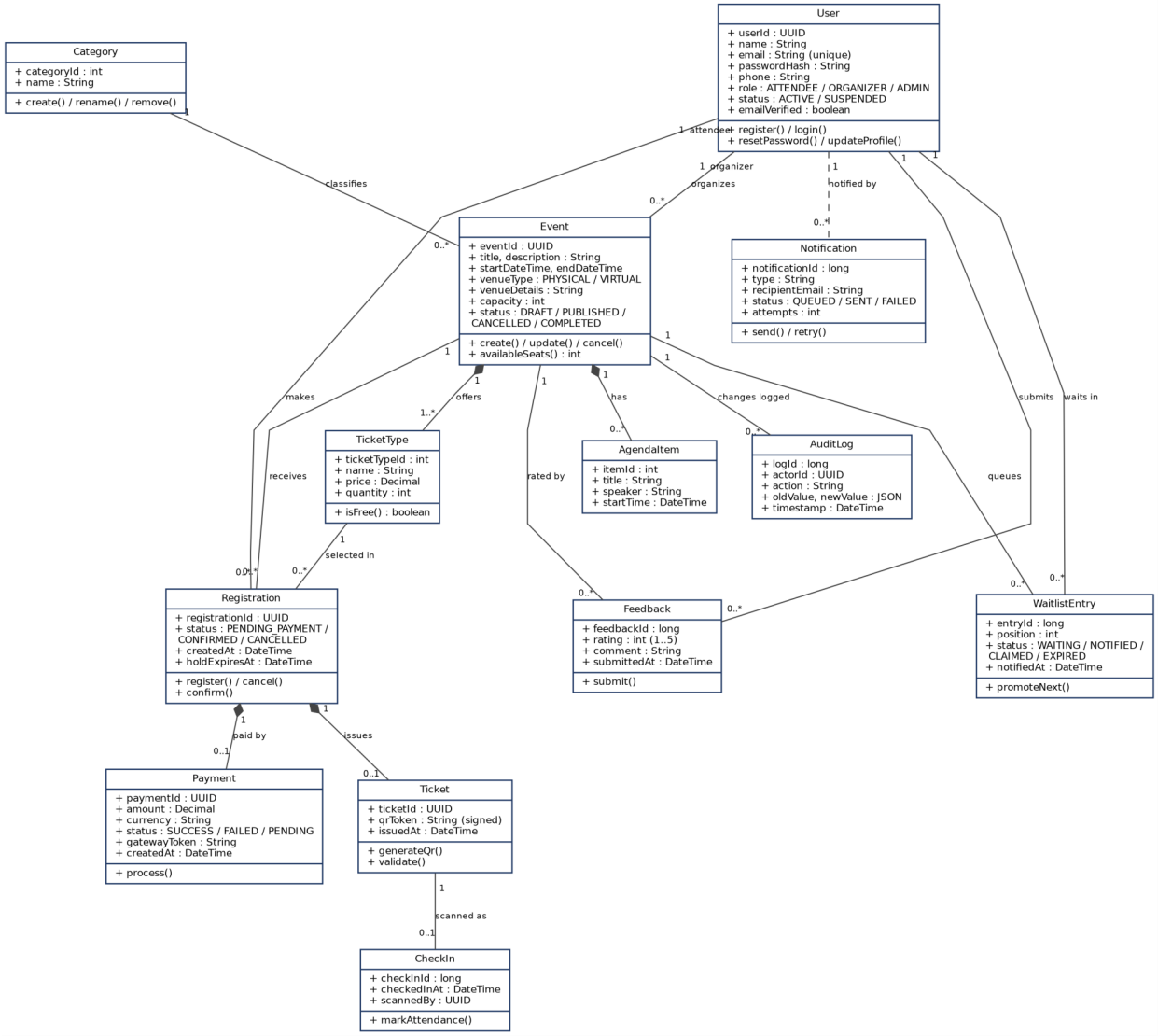


Figure 3.2 - EMRS domain class diagram (UML)

3.4 Component Descriptions

Component	Responsibility	Reqs	Collaborates with
Web Client (Responsive UI)	Responsive attendee, organizer and admin interface; WCAG 2.1 AA presentation and client-side validation; invokes the Backend REST API Gateway over REST/HTTPS.	NF-004	Backend REST API Gateway
QR Check-in Scanner	Camera-equipped phone/tablet interface used by organizer staff to scan e-ticket QR codes and submit check-in requests through the REST API.	F-016	Backend REST API Gateway, CheckInModule

Component	Responsibility	Reqs	Collaborates with
Backend REST API Gateway	Single REST entry point for the client components. Enforces HTTPS/TLS 1.2+, JWT validation, RBAC, rate limiting and CAPTCHA before dispatching requests to the business modules.	F-002, SEC-002, SEC-003, SEC-004	Web Client, QR Check-in Scanner, all Business Logic modules
EventModule	Creates, edits, cancels, browses/searches and displays event details; maintains event lifecycle and audit information; requests venue geocoding/map data when available.	F-005 to F-009	Backend REST API Gateway, RegistrationModule, NotificationModule, Maps / Geocoding API, Data Access Layer
AuthModule	Handles user registration, login/JWT issuance, password reset and profile operations. Passwords are stored using the approved password-hashing mechanism; admin accounts are provisioned separately.	F-001, F-002, F-003, F-004, SEC-001	Backend REST API Gateway, NotificationModule, Data Access Layer
RegistrationModule	Handles RSVP, capacity/duplicate checks, cancellation and waitlist management; coordinates payment, ticket generation and notifications.	F-010, F-011, F-012	EventModule, PaymentModule, TicketModule, NotificationModule, Data Access Layer
CheckInModule	Validates QR check-in requests, rejects invalid or duplicate scans, and records attendance.	F-016	QR Check-in Scanner, TicketModule, Data Access Layer
ReportingModule	Provides organizer dashboard statistics and CSV export for event reporting.	F-017	Data Access Layer
AdminModule	Provides administrative user/event moderation and category management functions.	F-018	AuthModule, EventModule, Backend REST API Gateway
FeedbackModule	Collects post-event ratings and feedback after an event concludes.	F-019	RegistrationModule, EventModule, Data Access Layer
NotificationModule	Handles notification creation and delivery for registration confirmation, event cancellation, waitlist updates, verification, password reset and configured reminders; delivery is asynchronous where applicable.	F-014	EventModule, RegistrationModule, AuthModule, Email/SMS Provider
PaymentModule	Processes simulated payment for paid registrations and records transaction status without storing card data.	F-013, SEC-005	RegistrationModule, Payment Gateway API
TicketModule	Generates e-tickets with signed QR tokens after confirmed registration and validates ticket tokens for check-in.	F-015	RegistrationModule, CheckInModule

Component	Responsibility	Reqs	Collaborates with
Maps / Geocoding API	External service used to obtain venue geocoding/map information for event details; the event view can fall back to the venue address when the service is unavailable.	F-009	EventModule
Email/SMS Provider	External notification delivery service used by NotificationModule. The current SRS specifies e-mail notifications; the UML boundary represents the provider interface.	F-014	NotificationModule
Payment Gateway API	External/simulated payment endpoint represented by the payment interface. The current release uses simulated payment processing and stores only permitted transaction tokens/references.	F-013, SEC-005	PaymentModule
Data Access Layer	Persistence boundary for the business modules. Uses repository/ORM operations and transaction management to read and update EMRS data.	NF-001, NF-003	Business Logic Layer, Relational Database
Relational Database	Persistent system of record for users, events, registrations, payments, tickets, notifications, feedback, audit information and related EMRS data.	NF-001, NF-002, NF-003	Data Access Layer

3.5 Chosen Architecture Pattern and Rationale

Chosen pattern: layered architecture implemented as a modular monolith. The UML component diagram is organised into a UI / Presentation Layer, a Backend REST API Gateway, a Business Logic Layer, a Data Access Layer, and external service interfaces. The Business Logic Layer is divided into feature modules (AuthModule, EventModule, RegistrationModule, CheckInModule, ReportingModule, AdminModule, FeedbackModule, NotificationModule, PaymentModule and TicketModule). The Backend REST API Gateway provides a single REST entry point and centralises transport and request-security controls shown in the UML diagram: HTTPS/TLS 1.2+, JWT validation, RBAC, rate limiting and CAPTCHA. Data persistence is isolated in the Data Access Layer, which connects to the relational database. External services are represented at the architecture boundary as the Maps / Geocoding API, Email/SMS Provider and Payment Gateway API. This structure keeps responsibilities separated while matching the component boundaries and interfaces shown in Figure 3.1.

Alternatives considered:

Alternative	Verdict	Reason
Microservices	Rejected	Overly complex for the scale and team size: needs service discovery, distributed transactions across registration, payment and ticketing, and heavier operations. Scalability (NF-003) is met by replicating the stateless monolith.
Server-rendered monolith (no API)	Rejected	Weaker separation of UI and logic, harder to test and to reuse the same API for the scanner UI; less suited to a responsive mobile-first experience.

Alternative	Verdict	Reason
Serverless functions	Rejected	Cold starts risk the 2-second target (NF-001), connection-pool limits with relational databases, and harder local testing.
Event-driven messaging (partially adopted)	Used only for notifications	Asynchronous persisted notifications keep requests fast and resilient (ADR-006); a full message broker is unnecessary at this scale.

Architecture Decision Records

ADR	Context and decision	Consequences
ADR-001 Layered modular monolith Status: Accepted	**Context:** four-person team, fixed deadline, moderate scale, need for testability (NF-005). **Decision:** one deployable Spring Boot application; layers Controller → Service → Repository; package-by-feature modules that interact via service interfaces only.	+ Simple deployment; ACID transactions across registration, payment and ticket issue; easy unit and integration testing. - Whole application is the scaling unit (acceptable because it is stateless); module boundaries must be enforced (e.g., architecture tests).
ADR-002 REST/JSON over HTTPS with SPA front-end Status: Accepted	**Context:** responsive UI for three roles; reusable API for the scanner. **Decision:** React SPA consumes a versioned REST API (`/api/v1`); contract documented with OpenAPI/Swagger.	+ Clear contract, independent UI and API testing. - Validation rules exist on client and server (server is authoritative).
ADR-003 Stateless JWT authentication Status: Accepted	**Context:** F-002 requires a JWT; NF-003 favours horizontal scaling. **Decision:** signed JWT (role and user id claims, 60-minute expiry) sent as a Bearer header; no server-side session.	+ No session store; any instance can serve any request. - Revocation is harder: mitigated by short expiry and a cached account-status check so suspended users (F-018) are rejected quickly.
ADR-004 MySQL with transactional capacity control Status: Accepted	**Context:** F-010 must prevent overselling and duplicate registration under concurrency. **Decision:** InnoDB transactions; lock the event row (`SELECT ... FOR UPDATE`) while counting seats; unique constraint on (event, user) for active registrations.	+ Correct under concurrent requests. - Row-lock contention on very popular events; mitigated by keeping the transaction short.
ADR-005 Payment behind a gateway adapter; tokens only Status: Accepted	**Context:** F-013 is simulated; SEC-005 forbids storing card data. **Decision:** `PaymentGateway` interface with a `SimulatedPaymentGateway` (outcome chosen by test token); only token, reference and status are stored.	+ Card data never reaches EMRS servers; a real PCI-DSS gateway can be plugged in later; QA can force each outcome. - Simulation does not exercise real gateway failure modes.
ADR-006 Asynchronous, persisted notifications with retry Status: Accepted	**Context:** e-mail is third-party and can be slow or down (SRS §2.5). **Decision:** Notification module stores each message (QUEUED) and a worker sends it via the Email Adapter; 3 retries with exponential back-off; business transactions never roll back because of e-mail failure.	+ Registration stays fast (NF-001) and resilient. - Delivery is eventual; failed messages must be monitored (§4.4).

ADR	Context and decision	Consequences
ADR-007 Signed opaque QR token Status: Accepted	**Context:** F-015/F-016 need unique, unforgeable tickets; QR codes are easy to photograph. **Decision:** QR encodes ticket id + random nonce + HMAC-SHA256 signature; no personal data; server validates and a unique constraint allows one check-in per ticket.	+ Forgery is detectable; no personal data leaks from a QR image; duplicate scans rejected. - Check-in needs connectivity (offline check-in is out of scope).

3.6 Technology Stack & Data Stores

The stack below is the proposed implementation of the layers shown in Figure 3.1. The Web Client and QR Check-in Scanner use the REST API; the Backend REST API Gateway provides the common security and routing boundary; the Business Logic Layer contains the feature modules; the Data Access Layer handles persistence; and the external provider components represent integration boundaries.

Concern	Technology	Rationale
Front-end	React SPA with a responsive CSS framework; browser camera API for scanning	Responsive mobile-first UI; component reuse across roles (NF-004).
Back-end	Java 17, Spring Boot 3 (Spring Web, Spring Security, Spring Data JPA)	Mature layered framework; built-in security filters and transaction support.
Authentication	JWT (signed), bcrypt via Spring Security `BCryptPasswordEncoder` (cost 12)	F-002, SEC-001.
Rate limiting / CAPTCHA	Token-bucket limiter (e.g., Bucket4j); reCAPTCHA or hCaptcha	SEC-004.
QR codes	ZXing library; HMAC-SHA256 signed token	F-015, ADR-007.
E-mail	Spring Mail (SMTP) or a transactional e-mail API	F-014, SRS §3.3.
Payments	Simulated gateway adapter (tokenised interface)	F-013, SEC-005.
Maps (optional)	Maps/Geocoding API or OpenStreetMap links	F-009, SRS §3.3.
Database	MySQL 8 (InnoDB)	Relational, transactional, widely supported.
Caching	In-memory cache (e.g., Caffeine) for categories and public event lists, 60 s TTL	NF-001.
Transport / hosting	Nginx or cloud load balancer terminating TLS 1.2+; cloud VM or PaaS	SEC-002, NF-002.
Build and CI	Maven, Git, CI pipeline running tests on each commit	NF-005.
Testing	JUnit 5, Mockito, Spring Boot Test, REST Assured/Postman, Selenium or Cypress, JMeter, OWASP ZAP, axe/Lighthouse	Supports the STP test types: functional, integration, performance, security, accessibility.
Monitoring	Spring Boot Actuator + Micrometer; Prometheus/Grafana or cloud monitoring	NF-001, NF-002 (§4.4).

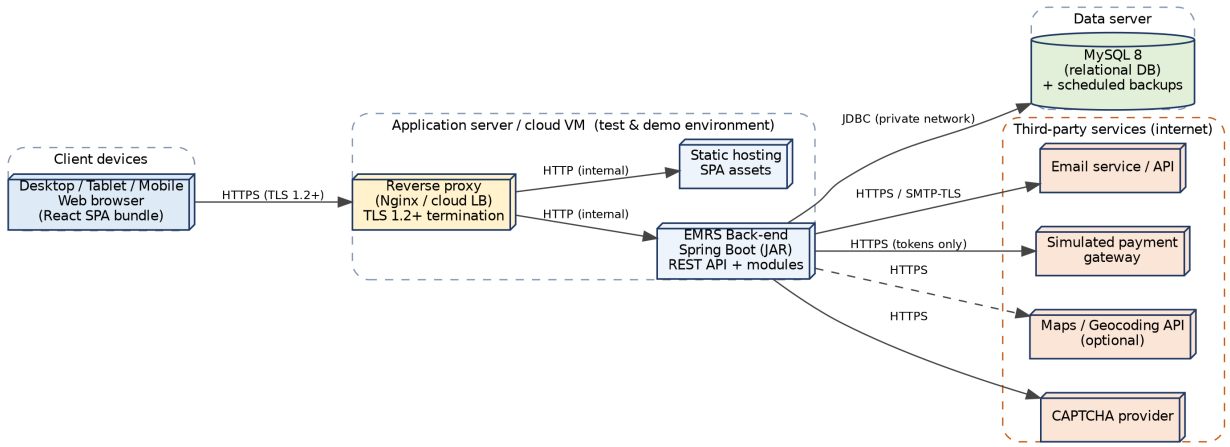


Figure 3.3 - EMRS deployment view

Data stores. A single MySQL database is the system of record. No card data and no file uploads are stored.

Table(s)	Content	Notes
`users`, `email_verification_tokens`, `password_reset_tokens`	Accounts, role, status, bcrypt hash; single-use time-limited tokens (24 h / 30 min).	`UNIQUE(email)`; hashes only, never plaintext.
`categories`, `events`, `ticket_types`, `agenda_items`	Event catalogue and ticket tiers.	Indexes on (status, start_datetime), category_id; full-text index on title and description for keyword search.
`registrations`, `waitlist_entries`	RSVP state, payment hold expiry, FIFO waitlist with offer expiry.	`UNIQUE(event_id, user_id)` for active registrations; capacity enforced in transaction.
`payments`	Amount, currency, status, gateway token/reference.	No card number, CVV or expiry.
`tickets`, `check_ins`	Unique e-ticket with signed token; scan record.	`UNIQUE(ticket_id)` in `check_ins` prevents double check-in.
`feedback`	Rating (1-5) and comment per attendee per event.	`UNIQUE(event_id, user_id)`.
`audit_log`	Actor, action, old/new values, timestamp for event edits, admin actions, payment status changes, check-ins.	Append-only; application DB user has no UPDATE/DELETE right on this table.
`notifications`	Queued/sent/failed e-mails, attempts.	Drives retry worker and failure monitoring.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
E-mail service unavailable or slow	Confirmations and cancellation notices delayed (F-014).	Persisted queue with 3 retries and back-off; failed messages visible on monitoring; registration unaffected (ADR-006).

Risk	Impact	Mitigation
Simulated payment differs from a real gateway	Defects appear when a real gateway is adopted.	Strict `PaymentGateway` interface and tokenised design; contract tests; documented limitation.
Concurrent registrations oversell an event	Capacity violated (F-010).	Row-level locking, short transactions, unique constraints, concurrency tests (ADR-004).
Peak load during popular registrations	Response time above 2 s (NF-001, NF-003).	Indexes, pagination, caching, pooled connections, stateless horizontal scaling; load tests in TC-Perf-01 and TC-Scale-01.
Brute-force login or bot registrations	Account takeover, fraudulent registrations.	Rate limiting, CAPTCHA, bcrypt, generic login errors (SEC-001, SEC-004).
Forged or shared QR tickets	Unauthorised entry.	HMAC-signed token, unique check-in constraint, organiser-only validation (ADR-007).
Personal-data exposure	Privacy breach, reputational damage.	TLS, RBAC, least-privilege DB user, no PII in QR or logs, secrets kept outside source control.
Maps/Geocoding or CAPTCHA provider outage	Venue map missing; sign-up blocked.	Map is optional (address text fallback); CAPTCHA failure treated as retry with rate limiting still active.
Database or server failure	System unavailable during demo (NF-002).	Health probes, automatic restart, scheduled backups, tested restore procedure.
Incomplete implementation or limited time	Requirements not testable.	Implement High-priority requirements first; modular structure allows incremental delivery; track via RTM.

3.8 Traceability to Requirements

Each SRS requirement is traced to the responsible component(s), the design reference in this document and the STP test case(s). All 29 requirements are covered.

Req ID	Requirement	Component(s)	Design reference	Test case(s)
EMRS-F-001	User Registration	AuthModule, NotificationModule, Backend REST API Gateway	§4.3.2, §4.3.3	TC-Auth-01, 02
EMRS-F-002	Login / Authentication	AuthModule, Backend REST API Gateway	Fig 4.1, §4.3.3, ADR-003	TC-Auth-03, 04
EMRS-F-003	Password Reset	AuthModule, NotificationModule	§4.3.2	TC-Auth-05
EMRS-F-004	User Profile Management	AuthModule	§3.4, §4.3.2	TC-Profile-01
EMRS-F-005	Create Event	EventModule, Data Access Layer	§4.3.3, §4.1	TC-Event-01, 02
EMRS-F-006	Edit / Update Event	EventModule, Data Access Layer	§4.3.2, §4.4	TC-Event-03
EMRS-F-007	Cancel Event	EventModule, NotificationModule	§4.3.2, §4.1 (lifecycle)	TC-Event-04
EMRS-F-008	Browse / Search Events	EventModule, Data Access Layer	§4.3.2, §3.6	TC-Event-05

Req ID	Requirement	Component(s)	Design reference	Test case(s)
EMRS-F-009	View Event Details	EventModule, Maps / Geocoding API	§3.4, §4.5	TC-Event-06
EMRS-F-010	Register for Event (RSVP)	RegistrationModule, Data Access Layer	Fig 4.2, §4.3.3, ADR-004	TC-Reg-01, 02
EMRS-F-011	Cancel Registration	RegistrationModule, Data Access Layer	Fig 4.3, §4.3.2	TC-Reg-03
EMRS-F-012	Waitlist Management	RegistrationModule, NotificationModule	Fig 4.3, §4.1	TC-Reg-04
EMRS-F-013	Payment Processing	PaymentModule, Payment Gateway API	Figure 3.1, §4.3.3, ADR-005	TC-Pay-01, 02
EMRS-F-014	Notifications & Reminders	NotificationModule, Email/SMS Provider	Figure 3.1, ADR-006, §4.3.4, §4.4	TC-Notif-01, 02
EMRS-F-015	e-Ticket / QR Code Generation	TicketModule	Figure 3.1, ADR-007	TC-QR-01
EMRS-F-016	Event Check-in	CheckInModule, TicketModule, QR Check-in Scanner	Figure 3.1, §4.3.3	TC-QR-02
EMRS-F-017	Organizer Dashboard & Reports	ReportingModule, Data Access Layer	§4.3.2, §4.5	TC-Report-01
EMRS-F-018	Admin User & Event Moderation	AdminModule, AuthModule, EventModule	§4.3.2	TC-Admin-01, 02
EMRS-F-019	Post-Event Feedback	FeedbackModule, RegistrationModule, EventModule	§4.3.2	TC-Feedback-01
EMRS-NF-001	Performance (95% < 2 s)	Data Access Layer, NotificationModule	§3.6, §4.4	TC-Perf-01
EMRS-NF-002	Availability	Backend REST API Gateway, Data Access Layer, Relational Database	§3.6, §4.4	TC-Avail-01
EMRS-NF-003	Scalability	Backend REST API Gateway, Data Access Layer	ADR-001, 003, 004; §3.6	TC-Scale-01
EMRS-NF-004	Usability & Accessibility	Web Client (Responsive UI), QR Check-in Scanner	§4.5	TC-UX-01
EMRS-NF-005	Maintainability	Backend REST API Gateway, Business Logic Layer, Data Access Layer	§3.5, §4.1	TC-Maint-01
EMRS-SEC-001	Password hashing (bcrypt/Argon2)	AuthModule	§3.9, §4.4	TC-Sec-01
EMRS-SEC-002	TLS 1.2 or higher	Backend REST API Gateway	Figure 3.1, §3.9	TC-Sec-02

Req ID	Requirement	Component(s)	Design reference	Test case(s)
EMRS-SEC-003	RBAC on every endpoint	Backend REST API Gateway	§3.9, Figure 3.1, §4.3.3	TC-Sec-03
EMRS-SEC-004	Rate limiting and CAPTCHA	Backend REST API Gateway	§3.9, Figure 3.1, §4.3.3	TC-Sec-04
EMRS-SEC-005	No card data stored	PaymentModule, Payment Gateway API	ADR-005, §3.9	TC-Sec-05

3.9 Security Architecture

Assets: credentials, personal data (name, e-mail, contact details), payment transaction records, tickets and QR tokens, event data and audit logs. Trust boundaries: (1) Internet/UI clients to the Backend REST API Gateway (HTTPS/TLS 1.2+); (2) Business Logic Layer to the Data Access Layer and relational database (private access, least-privilege account); (3) application modules to the external Maps / Geocoding API, Email/SMS Provider and Payment Gateway API (HTTPS, with provider credentials held in environment configuration). Admin accounts are provisioned outside the public sign-up flow, and public registration accepts only the Attendee and Organizer roles.

Threat model (STRIDE):

Threat	EMRS scenario	Mitigation in the design	Req
Spoofing	Attacker logs in as another user, forges a JWT or creates bot accounts.	bcrypt-hashed passwords; signed JWT with expiry; e-mail verification; rate limiting and CAPTCHA on login/registration; generic "invalid credentials" message.	SEC-001, SEC-004, F-002
Tampering	Request or token altered in transit; price or capacity manipulated; QR forged.	TLS 1.2+; server-side validation and server-side price lookup; parameterised queries (JPA); HMAC-signed QR tokens; JWT signature check; audit log of event changes.	SEC-002, ADR-007
Repudiation	Organizer or admin denies changing/removing an event; disputed payment or check-in.	Append-only audit log (actor, action, before/after, time) for event edits, admin actions, payment status and check-ins.	F-006, F-018
Information disclosure	Eavesdropping, leaked database, verbose errors or logs expose personal or payment data.	TLS; hashed passwords; no card data stored; tokenised payments; generic error messages; no sensitive data in logs; no personal data in QR; RBAC and ownership checks.	SEC-001, 002, 005
Denial of service	Request flooding, scripted registrations, heavy searches.	Per-IP/account rate limiting; pagination limits; caching; connection pooling; timeouts on third-party calls; health monitoring and alerts.	SEC-004, NF-001/003
Elevation of privilege	Attendee calls organizer/admin endpoints; organizer edits another organizer's event; self-assigns Admin role.	RBAC on every protected endpoint plus ownership checks; Admin role cannot be chosen at registration; deny-by-default authorization.	SEC-003

Security requirements to design mechanisms:

Requirement	Design mechanism	Verified by
EMRS-SEC-001 Salted hash, no plaintext	bcrypt (cost 12, per-password salt); passwords and hashes excluded from logs and API responses; password-reset tokens single-use.	TC-Sec-01
EMRS-SEC-002 TLS 1.2+	TLS terminated at the reverse proxy; TLS 1.0/1.1 disabled; HTTP redirected to HTTPS; HSTS header.	TC-Sec-02
EMRS-SEC-003 RBAC	Roles ATTENDEE, ORGANIZER, ADMIN enforced by method/endpoint security rules; deny by default; ownership check for organizer resources.	TC-Sec-03
EMRS-SEC-004 Rate limiting and CAPTCHA	Backend REST API Gateway	TC-Sec-04
EMRS-SEC-005 No card data	Client sends a gateway token only; Payment module persists token reference and status; no card fields in schema, logs or API.	TC-Sec-05

Additional measures (OWASP-aligned):

- Input validation on every DTO (bean validation) and output encoding in the SPA to prevent injection and cross-site scripting; Content-Security-Policy and other security headers.
- Authorization uses a Bearer header rather than cookies, so cross-site request forgery does not apply to the API.
- Secrets (JWT signing key, API keys, database password) are supplied by environment configuration, never committed to source control.
- Least-privilege database account; dependency vulnerability scanning in CI; security tests (OWASP ZAP) before acceptance (STP §5.5).

4. Design

4.1 Design Overview

EMRS is designed as layered modules for separation of concerns and easier maintenance. Each feature module follows the same internal structure, which keeps the code predictable and testable (NF-005):

Layer	Responsibility	Rules
Controller (REST)	Maps HTTP to DTOs, validates input, returns the standard response/error envelope.	No business logic; no repository access.
Service	Business rules, transactions, authorization checks that depend on data (ownership), orchestration of other modules through their service interfaces.	Only layer that begins transactions; calls adapters through interfaces.
Repository	Persistence via Spring Data JPA; query methods and locking.	Accessed only by the service of the same module.
Adapter (port)	Isolates e-mail, payment, maps and CAPTCHA providers.	Interface in the module, implementation replaceable (real, simulated, test stub).

Package structure: com.emrs.<module>.{controller, service, repository, domain, dto} with shared com.emrs.security, com.emrs.common (error handling, logging) and integration interfaces for the external Maps / Geocoding, Email/SMS and Payment Gateway services.

Design patterns used:

Pattern	Where used	Benefit
Layered architecture, Service Layer	All modules	Separation of concerns, testability.
Repository	Data Access Layer	Hides persistence details; easy to mock.
DTO	Controllers	Stable API contract independent of entities; prevents exposing internal fields such as password hashes.
Adapter / Ports and Adapters	Email, Payment Gateway, Maps, CAPTCHA	Third-party independence and test stubs.
Strategy	Payment gateway implementations (simulated / real)	Swap behaviour without changing Payment module.
Observer / domain events	Registration confirmed, event cancelled, slot available to Notification	Loose coupling between business modules and notifications.
State	Event, Registration, Payment and Waitlist lifecycles	Explicit allowed transitions (table below).

Entity lifecycles:

Entity	States	Allowed transitions and rules
Event	DRAFT, PUBLISHED, CANCELLED, COMPLETED	DRAFT → PUBLISHED; PUBLISHED → CANCELLED (organizer, notifies all registered attendees) or → COMPLETED (after end time). Edits allowed only while PUBLISHED/DRAFT and before the start time; every edit is audit-logged.
Registration	PENDING_PAYMENT, CONFIRMED, CANCELLED	Free event: created directly as CONFIRMED. Paid event: PENDING_PAYMENT (15-minute hold) → CONFIRMED on payment SUCCESS; hold expiry or user cancel → CANCELLED. CONFIRMED → CANCELLED before event start frees the slot.
Payment	PENDING, SUCCESS, FAILED	PENDING → SUCCESS or FAILED (reconciliation job); a FAILED payment may be retried while the hold is valid.
Waitlist entry	WAITING, NOTIFIED, CLAIMED, EXPIRED	WAITING → NOTIFIED when a slot opens (24-hour claim window) → CLAIMED when the attendee registers, or EXPIRED and the next entry is notified.

Capacity rule (F-010, F-012): available seats = capacity − (CONFIRMED registrations + non-expired PENDING_PAYMENT holds + non-expired NOTIFIED waitlist offers). The calculation and the insert happen inside one transaction while the event row is locked (ADR-004), and `UNIQUE(event_id, user_id)` blocks duplicates.

Scheduled jobs: release expired payment holds (every minute), expire waitlist offers and notify the next attendee, reconcile PENDING payments, and send optional event reminders 24 hours before start (F-014).

4.2 UML Sequence Diagrams

Four flows specific to EMRS are specified below. Blue dashed arrows are return messages, red arrows are error responses and shaded boxes are alternative flows.

Flow 1 - User login with rate limiting and JWT issue (F-002, SEC-001, SEC-003, SEC-004)

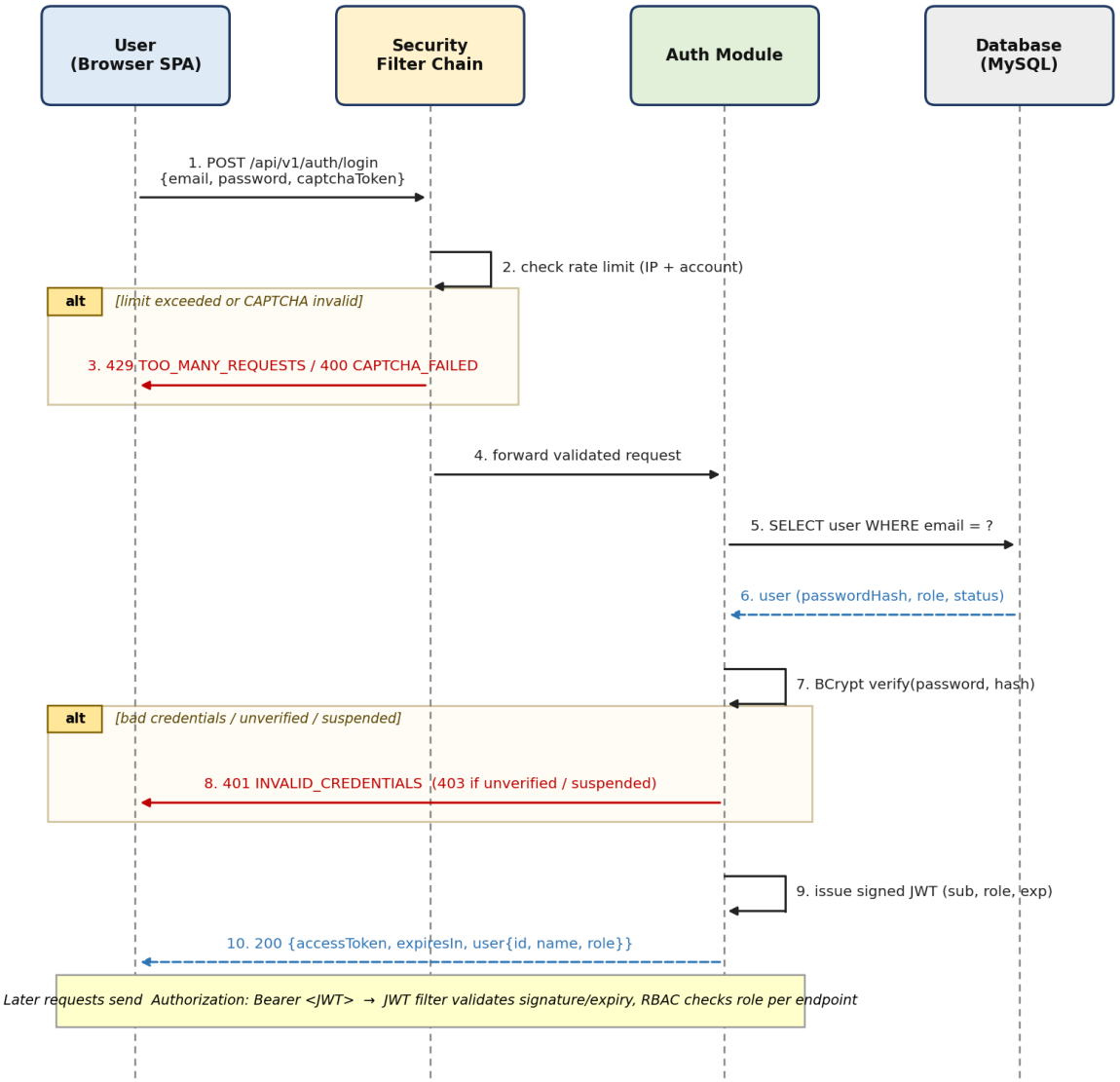


Figure 4.1 - Sequence diagram: user login

- The Backend REST API Gateway applies rate limiting and CAPTCHA checks before credential processing and enforces JWT/RBAC controls for protected requests.
- Credentials are verified against the bcrypt hash; failures return a generic 401 so that attackers cannot tell whether an e-mail exists.
- On success the Auth module issues a signed JWT; all later requests are authorised from it (RBAC).

Flow 2 - Register for a paid event and pay (F-010, F-013, F-014, F-015)

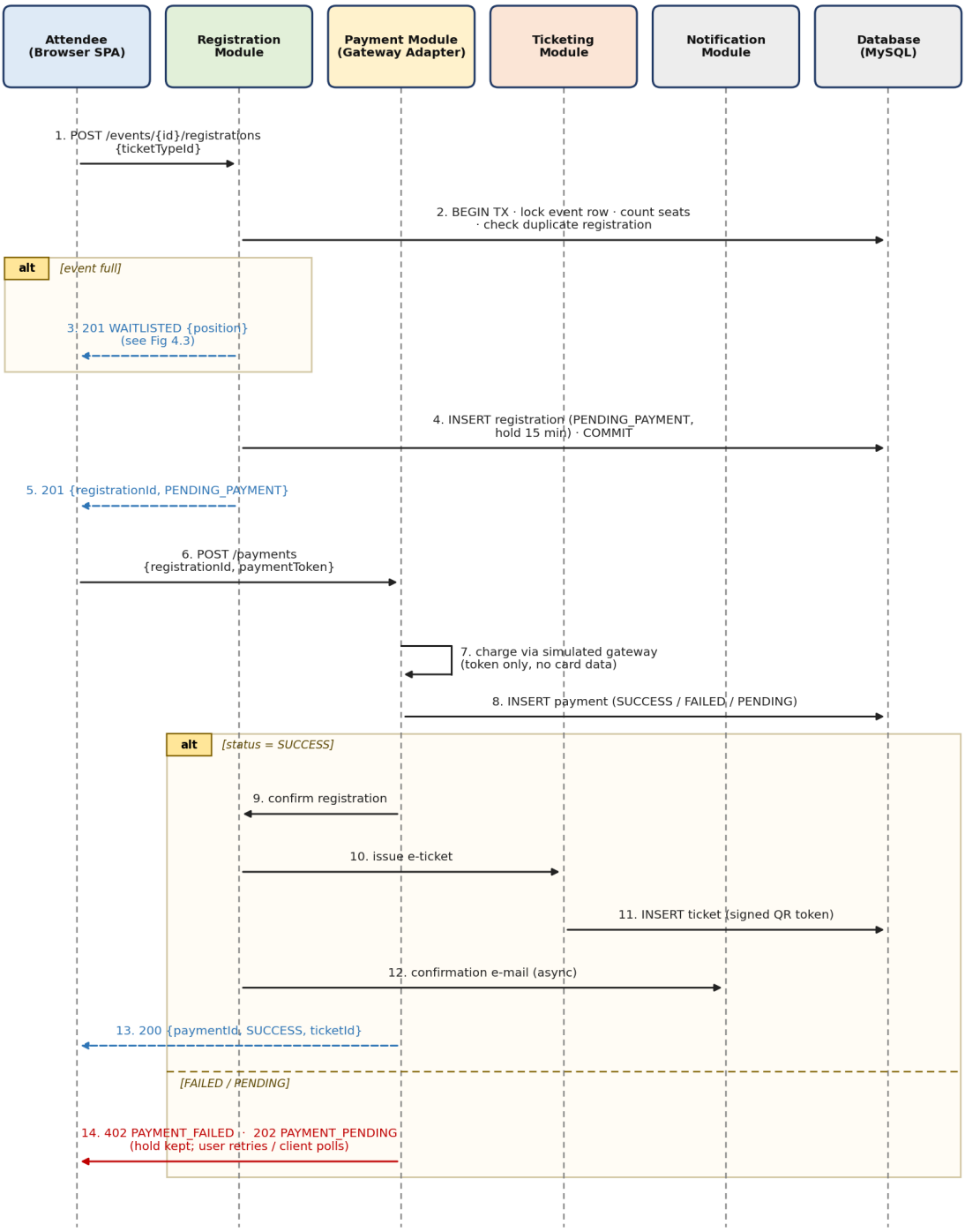


Figure 4.2 - Sequence diagram: register for a paid event, pay and receive an e-ticket

- Capacity check, duplicate check and insert are one transaction with the event row locked, so concurrent requests cannot oversell (ADR-004).
- The slot is held for 15 minutes in PENDING_PAYMENT while the attendee completes the multi-step checkout.

- Only a gateway token is sent; the simulated gateway returns SUCCESS, FAILED or PENDING and the status is stored (F-013, SEC-005).
- On SUCCESS the e-ticket with a signed QR token is issued and the confirmation e-mail is queued asynchronously (ADR-006); a FAILED payment keeps the hold so the user can retry.

Flow 3 - Cancel a registration and promote the waitlist (F-011, F-012, F-014)

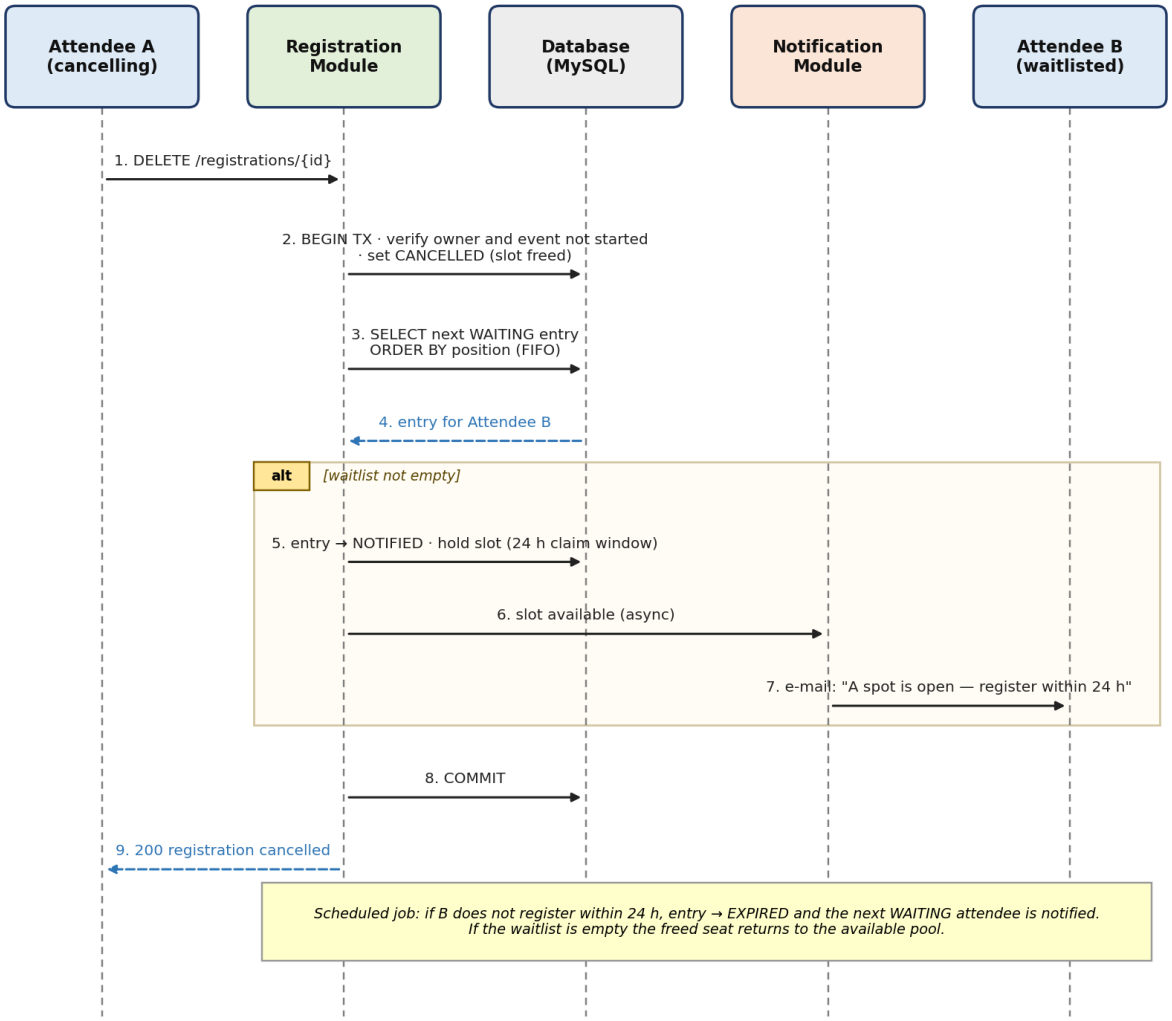


Figure 4.3 - Sequence diagram: cancel registration and notify the next waitlisted attendee

- Cancelling before the event starts frees the slot in the same transaction that selects the next waitlist entry (FIFO).
- The next attendee is notified and the slot is held for a 24-hour claim window; if unclaimed the entry expires and the next one is notified.
- Registration of an attendee who joined the waitlist when the event was full is created by the same flow as Figure 4.2, returning status WAITLISTED with the queue position.

Flow 4 - QR check-in at the venue (F-016, F-015, SEC-003)

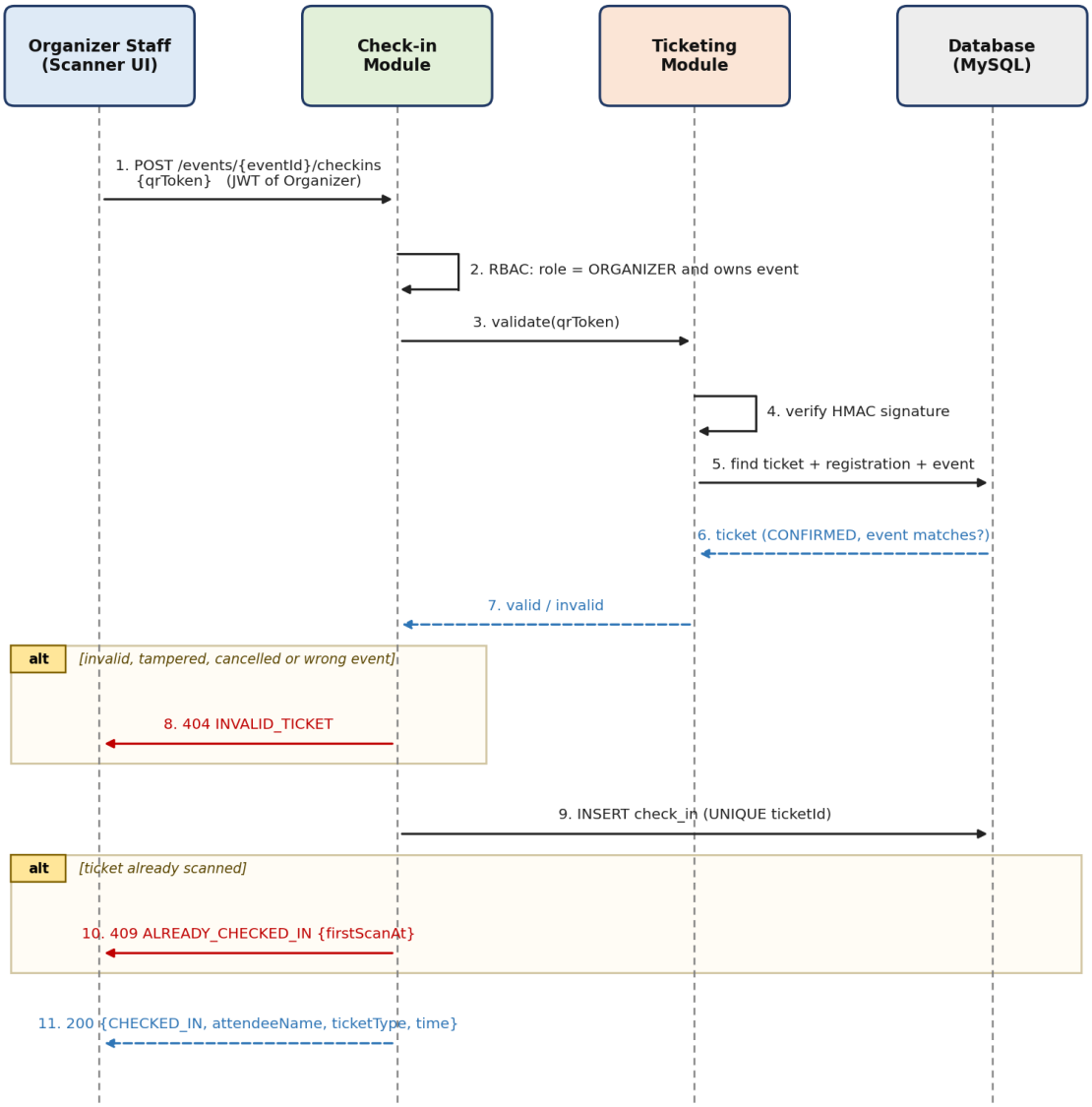


Figure 4.4 - Sequence diagram: QR ticket check-in

- Only the organizer who owns the event may validate tickets (RBAC plus ownership check).
- The signature, ticket status and event match are verified; invalid, tampered, cancelled or other-event tickets are rejected with 404.
- A unique constraint on the check-in table guarantees one check-in per ticket; a second scan returns 409 with the first scan time. Success is returned immediately so attendance is marked in real time.

4.3 API Design

Interfaces are defined for all feature modules (§4.3.2), in full detail for five components (§4.3.3), and for the internal adapter interfaces (§4.3.4).

4.3.1 Conventions

Base URL	`/api/v1`, HTTPS only (TLS 1.2+).
Format	JSON (UTF-8); `text/csv` for the report export. Dates/times in ISO-8601 UTC.
Authentication	`Authorization: Bearer <JWT>` on every endpoint except those marked Public.
Authorization	RBAC roles: Attendee, Organizer, Admin. "Owner" means the Organizer who owns the event.
Pagination	`page` (from 0) and `size` (default 20, max 100); list responses use `{items, page, size, total}`.
Errors	Standard error envelope and codes (§4.3.5).
Documentation	OpenAPI 3 specification served with Swagger UI.

4.3.2 Endpoint catalog

Module	Method and path	Purpose	Access	Req
Auth	`POST /auth/register`	Create account (name, email, password, role Attendee/Organizer); send verification e-mail	Public	F-001
	`GET /auth/verify-email?token=`	Verify e-mail address	Public	F-001
	`POST /auth/login`	Authenticate and receive JWT	Public	F-002
	`POST /auth/password-reset/request`	Request a reset link	Public	F-003
	`POST /auth/password-reset/confirm`	Set a new password with a valid token	Public	F-003
Profile	`GET /users/me`	View own profile	Any user	F-004
	`PUT /users/me`	Update name and contact information	Any user	F-004
Event	`POST /events`	Create event	Organizer	F-005
	`PUT /events/{id}`	Edit event before it starts (audit-logged)	Owner	F-006
	`GET /events/{id}/audit-log`	View change history	Owner, Admin	F-006
	`POST /events/{id}/cancel`	Cancel event and notify registered attendees	Owner	F-007
	`GET /events`	Search and filter by keyword, categoryId, from, to, location	Public	F-008
	`GET /events/{id}`	Event details: agenda, speakers, venue, map link	Public	F-009
	`GET /categories`	List categories	Public	F-008
Registration	`POST /events/{id}/registrations`	Register (RSVP) or join the waitlist	Attendee	F-010, F-012
	`DELETE /registrations/{id}`	Cancel registration; free slot; promote waitlist	Attendee (own)	F-011, F-012
	`GET /registrations/me`	List own registrations and waitlist status	Attendee	F-010 to 012
Payment	`POST /payments`	Pay for a PENDING_PAYMENT registration (simulated)	Attendee (own)	F-013

Module	Method and path	Purpose	Access	Req
	`GET /payments/{id}`	Transaction status	Attendee (own)	F-013
Ticketing	`GET /registrations/{id}/ticket`	Get e-ticket (ticket id and QR image)	Attendee (own)	F-015
Check-in	`POST /events/{id}/checkins`	Validate QR token and mark attendance	Owner	F-016
	`GET /events/{id}/checkins`	Live attendance list	Owner	F-016, F-017
Reporting	`GET /events/{id}/dashboard`	Registration count, revenue, attendance analytics	Owner	F-017
	`GET /events/{id}/report.csv`	Export report as CSV	Owner	F-017
Admin	`GET /admin/users`	List and search users	Admin	F-018
	`PUT /admin/users/{id}/suspend` and `/reinstate`	Suspend or reinstate a user	Admin	F-018
	`DELETE /admin/events/{id}`	Remove a policy-violating event	Admin	F-018
	`POST|PUT|DELETE /admin/categories[/{id}]`	Manage event categories	Admin	F-018
Feedback	`POST /events/{id}/feedback`	Submit rating and comment after the event concludes	Attendee (registered)	F-019
	`GET /events/{id}/feedback`	View feedback	Owner, Admin	F-019
System	`GET /actuator/health`	Health probe for monitoring	Internal	NF-002

4.3.3 Detailed interface definitions

Component 1 - Auth Module: register

Endpoint	`/api/v1/auth/register`
Method	`POST`
Access	Public; rate-limited and CAPTCHA protected (SEC-004)

Request	`{`
	`  "name": "string",`
	`  "email": "string",`
	`  "password": "string (min 8, upper, lower, digit)",`
	`  "role": "ATTENDEE | ORGANIZER",`
	`  "captchaToken": "string"`
	`}`

Response	**201 Created** `{ "userId": "uuid", "status": "PENDING_VERIFICATION" }`
Errors	400 VALIDATION_ERROR / CAPTCHA_FAILED / ROLE_NOT_ALLOWED (Admin cannot be chosen); 409 EMAIL_ALREADY_REGISTERED; 429 TOO_MANY_REQUESTS

Component 2 - Auth Module: login

Endpoint	`/api/v1/auth/login`
Method	`POST`
Access	Public; rate-limited; CAPTCHA required after repeated failures
Request	`{ "email": "string", "password": "string", "captchaToken": "string (when required)" }`
Response	**200 OK** `{ "accessToken": "<JWT>", "tokenType": "Bearer", "expiresIn": 3600, "user": { "id": "uuid", "name": "string", "role": "ATTENDEE" } }`
Errors	400 VALIDATION_ERROR / CAPTCHA_FAILED; 401 INVALID_CREDENTIALS; 403 EMAIL_NOT_VERIFIED / ACCOUNT_SUSPENDED; 429 TOO_MANY_REQUESTS

Component 3 - Event Module: create event

Endpoint	`/api/v1/events`
Method	`POST`
Access	Organizer (JWT)
Request	`{ "title": "string", "description": "string", "categoryId": 3, "startDateTime": "2026-11-20T09:00:00Z", "endDateTime": "2026-11-20T17:00:00Z", "venueType": "PHYSICAL | VIRTUAL", "venue": { "name": "string", "address": "string" }, "virtualLink": "https://... (virtual events)", "capacity": 200, "tickets": [{ "name": "General", "price": 0.00, "quantity": 150 }, { "name": "VIP", "price": 49.99, "quantity": 50 }], "agenda": [{ "title": "Keynote", "speaker": "string", "startTime": "..." }] }`
Response	**201 Created** `{ "eventId": "uuid", "status": "PUBLISHED" }`
Errors	400 VALIDATION_ERROR (missing field, end before start, start in the past, ticket quantities exceed capacity); 401 UNAUTHORIZED; 403 FORBIDDEN (not an Organizer); 404 CATEGORY_NOT_FOUND

Component 4 - Registration Module: register for an event

Endpoint	`/api/v1/events/{eventId}/registrations`
Method	`POST`
Access	Attendee (JWT)
Request	`{ "ticketTypeId": 12 }`

Response	**201 Created**
	`{`
	`  "registrationId": "uuid",`
	`  "status": "CONFIRMED | PENDING_PAYMENT | WAITLISTED",`
	`  "holdExpiresAt": "...  (paid events)",`
	`  "waitlistPosition": 4,`
	`  "ticketId": "uuid  (free events)"`
	`}`

Errors	400 VALIDATION_ERROR; 401 UNAUTHORIZED; 403 FORBIDDEN / ACCOUNT_SUSPENDED; 404 EVENT_NOT_FOUND; 409 DUPLICATE_REGISTRATION / EVENT_NOT_OPEN (cancelled or already started)

Component 5 - Payment Module: pay for a registration (simulated)

Endpoint	`/api/v1/payments`
Method	`POST`
Access	Attendee who owns the registration

Request	Only a token is sent - never card data (SEC-005).
	`{`
	`  "registrationId": "uuid",`
	`  "paymentToken": "tok_sim_success",`
	`  "currency": "USD"`
	`}`
	The amount is looked up on the server from the ticket price.

Response	**200 OK** (SUCCESS) or **202 Accepted** (PENDING)
	`{`
	`  "paymentId": "uuid",`
	`  "status": "SUCCESS | PENDING",`
	`  "registrationStatus": "CONFIRMED | PENDING_PAYMENT",`
	`  "ticketId": "uuid  (on SUCCESS)"`
	`}`

Errors	402 PAYMENT_FAILED (status FAILED recorded); 403 FORBIDDEN; 404 REGISTRATION_NOT_FOUND; 409 HOLD_EXPIRED / ALREADY_PAID

Component 6 - Check-in Module: validate a QR ticket

Endpoint	`/api/v1/events/{eventId}/checkins`
Method	`POST`
Access	Organizer who owns the event
Request	`{ "qrToken": "string (decoded QR content)" }`

Response	**200 OK**
	`{`
	`  "status": "CHECKED_IN",`
	`  "ticketId": "uuid",`
	`  "attendeeName": "string",`
	`  "ticketType": "VIP",`
	`  "checkedInAt": "..."`
	`}`

Errors	400 VALIDATION_ERROR; 401 UNAUTHORIZED; 403 FORBIDDEN (not the event owner); 404 INVALID_TICKET (bad signature, unknown, cancelled, other event); 409 ALREADY_CHECKED_IN (includes firstScanAt)

4.3.4 Internal component interfaces

Interface	Operations	Implementations	Req
RESTAPI	HTTPS REST/JSON request/response boundary from client components	Backend REST API Gateway	F-001 to F-019, NF-004, SEC-002
IEventMgmt	Event create, update, cancel, search and details operations	EventModule	F-005 to F-009
IAuth	Registration, login/JWT, password reset and profile operations	AuthModule	F-001 to F-004
IRegistration	RSVP, cancellation and waitlist operations	RegistrationModule	F-010 to F-012
ICheckIn	Validate QR ticket and record attendance	CheckInModule	F-016
IReporting	Organizer dashboard statistics and CSV export	ReportingModule	F-017
IModeration	User/event moderation and category management	AdminModule	F-018
IFeedback	Post-event rating and feedback operations	FeedbackModule	F-019
INotification	Create/send registration, cancellation, waitlist and account notifications	NotificationModule	F-014; also supports F-001, F-003, F-007, F-012
IPayment	Initiate and record simulated payment status	PaymentModule	F-013, SEC-005
ITicket	Issue and validate signed QR e-tickets	TicketModule	F-015, F-016
IGeocoding	Geocode venue address / obtain map information	Maps / Geocoding API	F-009
IEmailSMS	Deliver configured notification messages	Email/SMS Provider	F-014

Interface	Operations	Implementations	Req
IPaymentGateway	Process simulated payment requests using permitted token/reference data	Payment Gateway API	F-013, SEC-005
IRepository	Persistence operations exposed by the Data Access Layer	Data Access Layer	F-005, F-008, F-010, F-011, F-016, F-017, NF-001 to NF-003
ISQL	Database access for persistent EMRS records	Relational Database	NF-001 to NF-003

4.3.5 Error response format and codes

All errors use one envelope so that the client can show consistent messages:

Error body

```
{
  "timestamp": "2026-10-02T10:15:30Z",
  "status": 409,
  "errorCode": "DUPLICATE_REGISTRATION",
  "message": "You are already registered for this event.",
  "path": "/api/v1/events/{id}/registrations",
  "requestId": "b7f3c2...",
  "details": []
}
```

HTTP	errorCode examples	Used when
400	VALIDATION_ERROR, CAPTCHA_FAILED, ROLE_NOT_ALLOWED	Malformed or invalid input; details lists field errors.
401	UNAUTHORIZED, INVALID_CREDENTIALS, TOKEN_EXPIRED	Missing/invalid/expired JWT or wrong credentials.
402	PAYMENT_FAILED	Simulated payment declined.
403	FORBIDDEN, ACCOUNT_SUSPENDED, EMAIL_NOT_VERIFIED	Role or ownership check failed; suspended or unverified account.
404	EVENT_NOT_FOUND, REGISTRATION_NOT_FOUND, INVALID_TICKET	Resource does not exist or is not visible to the caller.
409	DUPLICATE_REGISTRATION, EVENT_NOT_OPEN, HOLD_EXPIRED, ALREADY_CHECKED_IN, EMAIL_ALREADY_REGISTERED	State conflict with the current resource state.
429	TOO_MANY_REQUESTS	Rate limit exceeded (SEC-004); includes `Retry-After`.
500 / 503	INTERNAL_ERROR, SERVICE_UNAVAILABLE	Unexpected failure or dependency (database) unavailable; no internal details exposed.

4.4 Error Handling, Logging & Monitoring

Error handling

- A global exception handler (`@RestControllerAdvice`) converts every exception to the standard envelope (§4.3.5). Stack traces, SQL and internal paths are never returned to clients.

- Layered validation: client-side checks for usability, bean validation on DTOs, business-rule checks in services, and database constraints as the last line of defence.
- Authentication failures return generic messages (no account enumeration); password reset always answers 202.
- Business transactions roll back completely on failure; no partial registrations or tickets.

Behaviour when external services fail:

Dependency	Failure	System behaviour
E-mail service	Timeout / error	Registration, cancellation etc. still succeed; message stays queued and is retried 3 times with exponential back-off; then marked FAILED and alerted.
Payment gateway	Timeout / declined	Declined: status FAILED, HTTP 402, hold kept for retry. Timeout: status PENDING (HTTP 202) and reconciliation job resolves it.
Maps/Geocoding API	Unavailable	Event details show the address text without a map; no error to the user.
CAPTCHA provider	Unavailable	Verification treated as failed with a retry message; rate limiting remains active.
Database	Unavailable	HTTP 503 SERVICE_UNAVAILABLE; health probe reports DOWN; alert raised.

Logging

- **Application logs:** structured JSON with timestamp, level, module, `requestId` (correlation id) and internal user id. Retained for 30 days.
- **Never logged:** passwords or hashes, JWTs, payment tokens, QR tokens, full e-mail addresses (masked), or any card data (SEC-001, SEC-005). No sensitive information appears in logs.
- **Audit log (database, append-only):** event create/edit/cancel with before and after values, admin suspensions, removals and category changes, payment status changes, check-ins, and failed-login lockouts (supports non-repudiation, §3.9).

Monitoring

Metric	Source	Alert threshold	Req
p95 response time	Micrometer timers	> 2 s over 5 minutes	NF-001
Uptime / health	`/actuator/health` probe every minute	Any failed probe for 2 minutes	NF-002
5xx error rate and transaction failure rate	Metrics counters	> 1% of requests	NF-002
DB connection-pool usage; registrations per minute	Micrometer	Pool > 80% used	NF-003
Failed or queued e-mails	`notifications` table	Any FAILED, or queue age > 15 min	F-014
Payment failure ratio	Payment counters	Unusual increase	F-013
Login failures and HTTP 429 spikes	Security filter metrics	Spike above baseline	SEC-004
Check-in latency	Timers on check-in endpoint	> 1 s	F-016

4.5 UX Design

The web UI is responsive and mobile-first, with distinct views for Attendee, Organizer and Admin (SRS §3.1, NF-004).

Role / view	Main screens	Design notes
Public / Attendee	Home and search with filters (keyword, category, date range, location); event details (agenda, speakers, venue map); registration and checkout; my registrations and e-tickets; feedback form.	Filters collapse into a drawer on mobile; clear "Event full - join waitlist" action; waitlist position shown; e-ticket QR is displayed full-width for scanning.
Organizer	Create/edit event form; my events; dashboard (registrations, revenue, attendance) with CSV export; QR scanner.	Form split into short steps with inline validation; KPI cards and charts; scanner shows a large green/red result banner with attendee name and a manual ticket-code fallback; live check-in counter.
Admin	User list with suspend/reinstate; event moderation; category management.	Confirmation dialogs for destructive actions; every action is audit-logged.

Registration and checkout flow (multi-step): 1 Select ticket → 2 Review and confirm → 3 Payment (simulated) → 4 Confirmation with e-ticket and e-mail. A progress indicator and a 15-minute hold countdown are shown; free events skip step 3.

Accessibility (WCAG 2.1 AA) and responsiveness:

- Responsive layouts for mobile (< 600 px), tablet (600-1024 px) and desktop (> 1024 px); touch targets of at least 44 px.
- Colour contrast of at least 4.5:1; information never conveyed by colour alone; text resizable to 200% without loss of function.
- Full keyboard navigation with visible focus; semantic HTML and ARIA labels; form labels and error messages announced to screen readers; alt text for images and QR codes.
- Consistent, plain-language error messages derived from the error codes in §4.3.5; loading and empty states on every screen.

4.6 Open Issues & Next Steps

#	Open issue	Proposed resolution	Owner
1	Final technology stack is proposed, not yet confirmed.	Confirm the stack in §3.6 before implementation; keep the component interfaces stable so product substitutions do not change the business modules.	Development Lead
2	SRS F-014 is titled "Notifications & Reminders" but specifies only confirmation and cancellation e-mails.	Confirm whether waitlist, verification, password-reset and reminder notifications shown in the UML are part of the accepted release scope.	Product Owner
3	Waitlist claim window and whether paid events use the same waitlist rules are not fully specified.	Confirm values and keep them configurable in application settings.	Product Owner

#	Open issue	Proposed resolution	Owner
4	Refunds for cancelled paid registrations/events are not specified.	Keep refunds out of the simulated-payment release unless the SRS is amended.	Product Owner
5	NF-002 and NF-003 have no numeric targets.	Agree measurable availability/scalability targets so TC-Avail-01 and TC-Scale-01 are objectively testable.	QA / Product Owner
6	Maps/Geocoding, Email/SMS and Payment Gateway providers are not fully selected.	Keep these integrations behind the interfaces represented in Figure 3.1; select concrete providers during implementation.	Development Lead

Future enhancements: real payment gateway with refunds; multi-factor authentication and social login; progressive web app with offline check-in; recurring events and organizer team roles; calendar (ICS) attachments; multi-language support.

5. Appendices

5.1 Glossary

Term	Definition
Adapter	Component that hides a third-party service behind an internal interface.
ADR	Architecture Decision Record.
Attendee / Organizer / Admin	The three user roles of EMRS (SRS §2.3).
Audit log	Append-only record of significant changes and actions.
Check-in	Validation of an e-ticket QR code at the venue and recording of attendance.
E-ticket	Digital ticket with a unique signed QR token, issued on confirmed registration.
Hold	Time-limited reservation of a seat while payment or waitlist claim is pending.
JWT	JSON Web Token used as session credential.
PCI-DSS	Payment Card Industry Data Security Standard.
RBAC	Role-Based Access Control.
RSVP	Registration of an attendee for an event.
STRIDE	Threat classification model (§3.9).
TLS	Transport Layer Security.
Waitlist	Ordered queue of attendees waiting for a seat in a full event.

5.2 References

- ISO/IEC/IEEE 42010 - Architecture description.
- EMRS Software Requirements Specification, v1.0 (03-09-2026).
- EMRS Software Test Plan, v1.0 (02-10-2026).
- OWASP Top 10 and OWASP Application Security Verification Standard (ASVS).
- NIST SP 800-160 - Systems Security Engineering.
- PCI DSS - Payment Card Industry Data Security Standard.
- W3C Web Content Accessibility Guidelines (WCAG) 2.1.
- RFC 7519 - JSON Web Token (JWT); RFC 5246 - TLS 1.2.
- OpenAPI Specification 3.0.

5.3 Tools

Purpose	Tools
Diagrams in this document	Graphviz (class and deployment diagrams); Python/Matplotlib (component and sequence diagrams). PlantUML or draw.io can be used to maintain editable copies.
API documentation	Swagger / OpenAPI
Development	Java 17, Spring Boot, Maven, React, MySQL, Git

Purpose	Tools
Testing (STP §6)	JUnit 5, Mockito, Spring Boot Test, Postman/REST Assured, Selenium or Cypress, JMeter, OWASP ZAP, axe/Lighthouse